

FASE 3 - Código refactorizado e informe final

Sistema de Reservas de Canchas Deportivas

1. Contexto

Este informe corresponde a la Fase 3 del proyecto integrador de Diseño de Software. La Fase 1 definió el diseño UML del sistema mediante el enfoque 4+1. La Fase 2 implementó una versión funcional mínima y documentó malos olores como método largo, lista de parámetros, código duplicado, obsesión por primitivos, números mágicos y clase con exceso de responsabilidades.

En esta fase se refactorizó el código manteniendo los casos de uso principales: crear reserva, consultar disponibilidad y cancelar reserva. El objetivo fue mejorar el diseño sin alterar el comportamiento observable del sistema.

2. Evidencia de pruebas

Comando usado después de cada refactorización:

```
python -m pytest
```

Resultado final: 7 pruebas aprobadas.

3. Refactorizaciones realizadas

1. Commit aeb0ff6 - Nivel: Métodos.

Refactorización: Se extrajeron métodos privados de validación en ReservationService.

Olor atendido: Método largo y código duplicado.

Resultado: Los métodos públicos quedaron más pequeños y legibles.

2. Commit a3b2574 - Nivel: Datos.

Refactorización: Se introdujo ReservationRequest como objeto de datos.

Olor atendido: Lista larga de parámetros.

Resultado: La creación puede recibir una estructura única y el método anterior se conserva como fachada.

3. Commit 2b5c36c - Nivel: Datos.

Refactorización: Se introdujo TimeSlot para inicio, duración, fin y solapamiento.

Olor atendido: Obsesión por primitivos y cálculos repetidos de fechas.

Resultado: La regla de solapamiento pasó del repositorio al dominio.

4. Commit 8afeb6d - Nivel: Clases/objetos y condicionales.

Refactorización: Se introdujo ReservationStatus y Reservation.cancel().

Olor atendido: Strings mágicos y manipulación externa del estado.

Resultado: La reserva controla su propia transición a cancelada.

5. Commit 6f56b32 - Nivel: Clases/objetos.

Refactorizacion: Se extrajo ReservationValidator.

Olor atendido: Clase de servicio con demasiadas responsabilidades.

Resultado: Las validaciones quedaron en una clase especializada e inyectable.

6. Commit 37af529 - Nivel: Condicionales.

Refactorizacion: Se agrego has_active_conflict() y Reservation.is_confirmed().

Olor atendido: Condicionales compuestos y consultas poco expresivas.

Resultado: El servicio expresa la regla como disponibilidad/conflicto y el repositorio encapsula el filtro.

4. Cobertura de los cuatro niveles solicitados

4.1 Nivel de metodos

create_reservation concentraba validaciones, construccion de objetos, verificacion de conflictos, persistencia y notificacion. Se extrajeron validaciones y se separo el flujo basado en ReservationRequest.

4.2 Nivel de clases/objetos

ReservationService conserva la orquestacion. ReservationValidator contiene reglas de entrada. Reservation encapsula la cancelacion mediante cancel().

4.3 Nivel de datos

ReservationRequest reduce la lista de parametros. TimeSlot representa el concepto de Horario definido en Fase 1 y concentra inicio, duracion, fin y solapamiento.

4.4 Nivel de condicionales

Reservation.is_confirmed() reemplaza comparaciones directas con literales. has_active_conflict() expresa la disponibilidad como una consulta booleana.

5. Evolucion del diseno

- Horario se implementa como TimeSlot, objeto de valor encargado de end_at y overlaps().
- Reserva aumento su cohesion porque controla su estado mediante cancel().
- ServicioReserva mantiene el rol de orquestador, pero ya no concentra todas las validaciones.
- ReservationValidator se agrego como colaborador especializado.
- ReservationStatus reemplaza literales de texto para evitar errores por strings magicos.
- ReservationRequest documenta los datos necesarios para crear una reserva.

El diagrama actualizado esta disponible en docs/diagrama_clases_actualizado_fase_3.svg y docs/diagrama_clases_actualizado_fase_3.puml.

6. Estado final

La implementacion final mantiene los tres casos de uso de la Fase 1 y conserva las pruebas unitarias aprobadas. La historia de Git contiene seis commits independientes de refactorizacion, lo que permite revisar cada diff antes/después y comprobar que el sistema se mantuvo funcional durante todo el proceso.

7. Comandos utiles

```
git log --oneline -6
python -m pytest
```

```
37af529 refactor: simplify reservation availability conditionals
6f56b32 refactor: extract reservation validator
8afeb6d refactor: encapsulate reservation status changes
2b5c36c refactor: introduce time slot value object
a3b2574 refactor: introduce reservation request object
aeb0ff6 refactor: extract reservation service validation methods
```